

valid [Uri](#) object, created using a feed URI string provided by the app. The [Uri](#) object is then passed to the [RetrieveFeedAsync](#) function to request the feed.

Assuming the desired feed content was returned, the example code iterates through each feed item, calling [displayCurrentItem](#) (which we define next), to display items and their contents as a list through the UI.

You must write code to handle exceptions when you call most asynchronous network methods. Your exception handler can retrieve more detailed information on the cause of the exception to better understand the failure and make appropriate decisions.

The [RetrieveFeedAsync](#) method throws an exception if a connection could not be established with the HTTP server or the [Uri](#) object does not point to a valid AtomPub or RSS feed. The JavaScript sample code uses an [onError](#) function to catch any exceptions and print out more detailed information on the exception if an error occurs.

C#

```
try
{
    // Although most HTTP servers do not require User-Agent header,
    // others will reject the request or return a different response if this
    // header is missing.
    // Use the setRequestHeader() method to add custom headers.
    client.SetRequestHeader("User-Agent", "Mozilla/5.0 (compatible; MSIE
10.0; Windows NT 6.2; WOW64; Trident/6.0)");
    feed = await client.RetrieveFeedAsync(uri);
    // Retrieve the title of the feed and store it in a string.
    string title = feed.Title.Text;
    // Iterate through each feed item.
    foreach (Windows.Web.Syndication.SyndicationItem item in feed.Items)
    {
        displayCurrentItem(item);
    }
}
catch (Exception ex)
{
    // Handle the exception here.
}
```

JavaScript

```
function onError(err) {
    WinJS.log && WinJS.log(err, "sample", "error");
    // Match error number with an ErrorStatus value.
    // Use Windows.Web.WebErrorStatus.getStatus() to retrieve HTTP error
    // status codes.
    var errorStatus =
    Windows.Web.Syndication.SyndicationError.getStatus(err.number);
```

```

        if (errorStatus ===
Windows.Web.Syndication.SyndicationErrorStatus.invalidXml) {
            displayLog("An invalid XML exception was thrown. Please make sure to
use a URI that points to a RSS or Atom feed.");
        }
    }
    // Retrieve and display feed at given feed address.
    function retrieveFeed(uri) {
        // Although most HTTP servers do not require User-Agent header,
        // others will reject the request or return a different response if this
        header is missing.
        // Use the setRequestHeader() method to add custom headers.
        client.setRequestHeader("User-Agent", "Mozilla/5.0 (compatible; MSIE
10.0; Windows NT 6.2; WOW64; Trident/6.0)");
        client.retrieveFeedAsync(uri).done(function (feed) {
            currentFeed = feed;
            WinJS.log && WinJS.log("Feed download complete.", "sample",
"status");
            var title = "(no title)";
            if (currentFeed.title) {
                title = currentFeed.title.text;
            }
            document.getElementById("CurrentFeedTitle").innerText = title;
            currentItemIndex = 0;
            if (currentFeed.items.size > 0) {
                displayCurrentItem();
            }
            // List the items.
            displayLog("Items: " + currentFeed.items.size);
        }, onError);
    }
}

```

In the previous step, [RetrieveFeedAsync](#) returned the requested feed content and the example code got to work iterating through available feed items. Each of these items is represented using a [SyndicationItem](#) object that contains all of the item properties and content afforded by the relevant syndication standard (RSS or Atom). In the following example we observe the **displayCurrentItem** function working through each item and displaying its content through various named UI elements.

C#

```

private void displayCurrentItem(Windows.Web.Syndication.SyndicationItem
item)
{
    string itemTitle = item.Title == null ? "No title" : item.Title.Text;
    string itemLink = item.Links == null ? "No link" :
item.Links.FirstOrDefault().ToString();
    string itemContent = item.Content == null ? "No content" :
item.Content.Text;
    //displayCurrentItem is continued below.
}

```

JavaScript

```
function displayCurrentItem() {
    var item = currentFeed.items[currentItemIndex];
    // Display item number.
    document.getElementById("Index").innerText = (currentItemIndex + 1) + "
of " + currentFeed.items.size;
    // Display title.
    var title = "(no title)";
    if (item.title) {
        title = item.title.text;
    }
    document.getElementById("ItemTitle").innerText = title;
    // Display the main link.
    var link = "";
    if (item.links.size > 0) {
        link = item.links[0].uri.absoluteUri;
    }
    var link = document.getElementById("Link");
    link.innerText = link;
    link.href = link;
    // Display the body as HTML.
    var content = "(no content)";
    if (item.content) {
        content = item.content.text;
    }
    else if (item.summary) {
        content = item.summary.text;
    }
    document.getElementById("WebView").innerHTML =
window.toStaticHTML(content);
    //displayCurrentItem is continued below.
```

As suggested earlier, the type of content represented by a [SyndicationItem](#) object will differ depending on the feed standard (RSS or Atom) employed to publish the feed. For example, an Atom feed is capable of providing a list of [Contributors](#), but an RSS feed is not. However, extension elements included in a feed item that are not supported by either standard (for example, Dublin Core extension elements) can be accessed using the [SyndicationItem.ElementExtensions](#) property and then displayed as demonstrated in the following example code.

C#

```
//displayCurrentItem continued.
string extensions = "";
foreach (Windows.Web.Syndication.SyndicationNode node in
item.ElementExtensions)
{
    string nodeName = node.NodeName;
    string nodeNamespace = node.NodeNamespace;
    string nodeValue = node.NodeValue;
```

```

        extensions += nodeName + "\n" + nodeNamespace + "\n" + nodeValue +
"\n";
    }
    this.listView.Items.Add(itemTitle + "\n" + itemLink + "\n" + itemContent
+ "\n" + extensions);
}

```

JavaScript

```

// displayCurrentItem function continued.
var bindableNodes = [];
for (var i = 0; i < item.elementExtensions.size; i++) {
    var bindableNode = {
        nodeName: item.elementExtensions[i].nodeName,
        nodeNamespace: item.elementExtensions[i].nodeNamespace,
        nodeValue: item.elementExtensions[i].nodeValue,
    };
    bindableNodes.push(bindableNode);
}
var dataList = new WinJS.Binding.List(bindableNodes);
var listView = document.getElementById("extensionsListView").winControl;
WinJS.UI.setOptions(listView, {
    itemDataSource: dataList.dataSource
});
}

```

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Background transfers

Article • 10/20/2022

Use the background transfer API to copy files reliably over the network. The background transfer API provides advanced upload and download features that run in the background during app suspension and persist beyond app termination. The API monitors network status and automatically suspends and resumes transfers when connectivity is lost, and transfers are also Data Sense-aware and Battery Sense-aware, meaning that download activity adjusts based on your current connectivity and device battery status. The API is ideal for uploading and downloading large files using HTTP(S). FTP is also supported, but only for downloads.

Background Transfer runs separately from the calling app and is primarily designed for long-term transfer operations for resources like video, music, and large images. For these scenarios, using Background Transfer is essential because downloads continue to progress even when the app is suspended.

If you are downloading small resources that are likely to complete quickly, you should use [HttpClient](#) APIs instead of Background Transfer.

Using `Windows.Networking.BackgroundTransfer`

How does the Background Transfer feature work?

When an app uses Background Transfer to initiate a transfer, the request is configured and initialized using [BackgroundDownloader](#) or [BackgroundUploader](#) class objects. Each transfer operation is handled individually by the system and separate from the calling app. Progress information is available if you want to give status to the user in your app's UI, and your app can pause, resume, cancel, or even read from the data while the transfer is occurring. The way transfers are handled by the system promotes smart power usage and prevents problems that can arise when a connected app encounters events such as app suspension, termination, or sudden network status changes.

Note

Due to per-app resource constraints, an app should not have more than 200 transfers (DownloadOperations + UploadOperations) at any given time. Exceeding that limit may leave the app's transfer queue in an unrecoverable state.

When an application is launched, it must call [AttachAsync](#) on all existing [DownloadOperation](#) and [UploadOperation](#) objects. Not doing this will cause the leak of already-completed transfers and will eventually render your use of the Background Transfer feature useless.

Performing authenticated file requests with Background Transfer

Background Transfer provides methods that support basic server and proxy credentials, cookies, and the use of custom HTTP headers (via [SetRequestHeader](#)) for each transfer operation.

How does this feature adapt to network status changes or unexpected shutdowns?

The Background Transfer feature maintains a consistent experience for each transfer operation when network status changes occur, by intelligently leveraging connectivity and carrier data-plan status information provided by the [Connectivity](#) feature. To define behavior for different network scenarios, an app sets a cost policy for each operation using values defined by [BackgroundTransferCostPolicy](#).

For example, the cost policy defined for an operation can indicate that the operation should be paused automatically when the device is using a metered network. The transfer is then automatically resumed (or restarted) when a connection to an "unrestricted" network has been established. For more information on how networks are defined by cost, see [NetworkCostType](#).

While the Background Transfer feature has its own mechanisms for handling network status changes, there are other general connectivity considerations for network-connected apps. Read [Leveraging available network connection information](#) for additional info.

Note For apps running on mobile devices, there are features that allow the user to monitor and restrict the amount of data that is transferred based on the type of connection, roaming status, and the user's data plan. Because of this, background transfers may be paused on the phone even when the [BackgroundTransferCostPolicy](#) indicates that the transfer should proceed.

The following table indicates when background transfers are allowed on the phone for each [BackgroundTransferCostPolicy](#) value, given the current state of the phone. You can use the [ConnectionCost](#) class to determine the phone's current state.

 Expand table

Device State	UnrestrictedOnly	Default	Always
Connected to WiFi	Allow	Allow	Allow
Metered Connection, not roaming, under data limit, on track to stay under limit	Deny	Allow	Allow
Metered Connection, not roaming, under data limit, on track to exceed limit	Deny	Deny	Allow
Metered Connection, roaming, under data limit	Deny	Deny	Allow
Metered Connection, over data limit. This state only occurs when the user enables "Restrict background data" in the Data Sense UI.	Deny	Deny	Deny

Uploading files

When using Background Transfer an upload exists as an [UploadOperation](#) that exposes a number of control methods that are used to restart or cancel the operation. App events (for example, suspension or termination) and connectivity changes are handled automatically by the system per [UploadOperation](#); uploads will continue during app suspension periods or pause and persist beyond app termination. Additionally, setting the [CostPolicy](#) property will indicate whether or not your app will start uploads while a metered network is being used for Internet connectivity.

The following examples will walk you through the creation and initialization of a basic upload and how to enumerate and reintroduce operations persisted from a previous app session.

Uploading a single file

The creation of an upload begins with [BackgroundUploader](#). This class is used to provide the methods that enable your app to configure the upload before creating the resultant [UploadOperation](#). The following example shows how to do this with the required [Uri](#) and [StorageFile](#) objects.

Identify the file and destination for the upload

Before we can begin with the creation of an **UploadOperation**, we first need to identify the URI of the location to upload to, and the file that will be uploaded. In the following example, the *uriString* value is populated using a string from UI input, and the *file* value using the **StorageFile** object returned by a **PickSingleFileAsync** operation.

JavaScript

```
function uploadFile() {
    var filePicker = new Windows.Storage.Pickers.FileOpenPicker();
    filePicker.fileTypeFilter.replaceAll(["*"]);

    filePicker.pickSingleFileAsync().then(function (file) {
        if (!file) {
            printLog("No file selected");
            return;
        }

        var upload = new UploadOp();
        var uriString = document.getElementById("serverAddressField").value;
        upload.start(uriString, file);

        // Store the upload operation in the uploadOps array.
        uploadOperations.push(upload);
    });
}
```

Create and initialize the upload operation

In the previous step the *uriString* and *file* values are passed to an instance of our next example, **UploadOp**, where they are used to configure and start the new upload operation. First, *uriString* is parsed to create the required **Uri** object.

Next, the properties of the provided **StorageFile** (*file*) are used by **BackgroundUploader** to populate the request header and set the *SourceFile* property with the **StorageFile** object. The **SetRequestHeader** method is then called to insert the file name, provided as a string, and the **StorageFile.Name** property.

Finally, **BackgroundUploader** creates the **UploadOperation** (*upload*).

JavaScript

```
function UploadOp() {
    var upload = null;
    var promise = null;

    this.start = function (uriString, file) {
        try {

            var uri = new Windows.Foundation.Uri(uriString);
```

```

    var uploader = new
Windows.Networking.BackgroundTransfer.BackgroundUploader();

    // Set a header, so the server can save the file (this is
specific to the sample server).
    uploader.setRequestHeader("Filename", file.name);

    // Create a new upload operation.
    upload = uploader.createUpload(uri, file);

    // Start the upload and persist the promise to be able to cancel
the upload.
    promise = upload.startAsync().then(complete, error, progress);
    } catch (err) {
        displayError(err);
    }
};
// On application activation, reassign callbacks for a upload
// operation persisted from previous application state.
this.load = function (loadedUpload) {
    try {
        upload = loadedUpload;
        promise = upload.attachAsync().then(complete, error, progress);
    } catch (err) {
        displayError(err);
    }
};
}

```

Note the asynchronous method calls defined using JavaScript promises. Looking at a line from the last example:

JavaScript

```
promise = upload.startAsync().then(complete, error, progress);
```

The async method call is followed by a `then` statement which indicates methods, defined by the app, that are called when a result from the async method call is returned. For more information on this programming pattern, see [Asynchronous programming in JavaScript using promises](#).

Uploading multiple files

Identify the files and destination for the upload

In a scenario involving multiple files transferred with a single **UploadOperation**, the process begins as it usually does by first providing the required destination URI and local file information. Similar to the example in the previous section, the URI is provided

as a string by the end-user and [FileOpenPicker](#) can be used to provide the ability to indicate files through the user interface as well. However, in this scenario the app should instead call the [PickMultipleFilesAsync](#) method to enable the selection of multiple files through the UI.

JavaScript

```
function uploadFiles() {
    var filePicker = new Windows.Storage.Pickers.FileOpenPicker();
    filePicker.fileTypeFilter.replaceAll(["*"]);

    filePicker.pickMultipleFilesAsync().then(function (files) {
        if (files === 0) {
            printLog("No file selected");
            return;
        }

        var upload = new UploadOperation();
        var uriString =
document.getElementById("serverAddressField").value;
        upload.startMultipart(uriString, files);

        // Persist the upload operation in the global array.
        uploadOperations.push(upload);
    });
}
```

Create objects for the provided parameters

The next two examples use code contained in a single example method, **startMultipart**, which was called at the end of the last step. For the purpose of instruction the code in the method that creates an array of [BackgroundTransferContentPart](#) objects has been split from the code that creates the resultant [UploadOperation](#).

First, the URI string provided by the user is initialized as a [Uri](#). Next, the array of [IStorageFile](#) objects (**files**) passed to this method is iterated through, each object is used to create a new [BackgroundTransferContentPart](#) object which is then placed in the **contentParts** array.

JavaScript

```
upload.startMultipart = function (uriString, files) {
    try {
        var uri = new Windows.Foundation.Uri(uriString);
        var uploader = new
Windows.Networking.BackgroundTransfer.BackgroundUploader();

        var contentParts = [];
        files.forEach(function (file, index) {
```

```

        var part = new
Windows.Networking.BackgroundTransfer.BackgroundTransferContentPart("File" +
index, file.name);
        part.setFile(file);
        contentParts.push(part);
    });

```

Create and initialize the multi-part upload operation

With our `contentParts` array populated with all of the `BackgroundTransferContentPart` objects representing each `IStorageFile` for upload, we are ready to call `CreateUploadAsync` using the `Uri` to indicate where the request will be sent.

JavaScript

```

        // Create a new upload operation.
        uploader.createUploadAsync(uri, contentParts).then(function
(uploadOperation) {

            // Start the upload and persist the promise to be able to
cancel the upload.
            upload = uploadOperation;
            promise = uploadOperation.startAsync().then(complete, error,
progress);
        });

        } catch (err) {
            displayError(err);
        }
    };

```

Restarting interrupted upload operations

On completion or cancellation of an `UploadOperation`, any associated system resources are released. However, if your app is terminated before either of these things can occur, any active operations are paused and the resources associated with each remain occupied. If these operations are not enumerated and re-introduced to the next app session, they will not be completed and will continue to occupy device resources.

1. Before defining the function that enumerates persisted operations, we need to create an array that will contain the `UploadOperation` objects that it will return:

JavaScript

```

var uploadOperations = [];

```

2. Next we define the function that enumerates persisted operations and stores them in our array. Note that the **load** method called to re-assign callbacks to the **UploadOperation**, should it persist through app termination, is in the UploadOp class we define later in this section.

JavaScript

```
function
Windows.Networking.BackgroundTransfer.BackgroundUploader.getCurrentUploadsAsync() {
    .then(function (uploads) {
        for (var i = 0; i < uploads.size; i++) {
            var upload = new UploadOp();
            upload.load(uploads[i]);
            uploadOperations.push(upload);
        }
    });
};
```

Downloading files

When using Background Transfer, each download exists as a **DownloadOperation** that exposes a number of control methods used to pause, resume, restart, and cancel the operation. App events (for example, suspension or termination) and connectivity changes are handled automatically by the system per **DownloadOperation**; downloads will continue during app suspension periods or pause and persist beyond app termination. For mobile network scenarios, setting the **CostPolicy** property will indicate whether or not your app will begin or continue downloads while a metered network is being used for Internet connectivity.

If you are downloading small resources that are likely to complete quickly, you should use **HttpClient** APIs instead of Background Transfer.

The following examples will walk you through the creation and initialization of a basic download, and how to enumerate and reintroduce operations persisted from a previous app session.

Configure and start a Background Transfer file download

The following example demonstrates how strings representing a URI and a file name can be used to create a **Uri** object and the **StorageFile** that will contain the requested file. In this example, the new file is automatically placed in a pre-defined location. Alternatively, **FileSavePicker** can be used allow users to indicate where to save the file on the device.

Note that the **load** method called to re-assign callbacks to the **DownloadOperation**, should it persist through app termination, is in the DownloadOp class defined later in this section.

JavaScript

```
function DownloadOp() {
    var download = null;
    var promise = null;
    var imageStream = null;

    this.start = function (uriString, fileName) {
        try {
            // Asynchronously create the file in the pictures folder.

            Windows.Storage.KnownFolders.picturesLibrary.createFileAsync(fileName,
            Windows.Storage.CreationCollisionOption.generateUniqueName).done(function
            (newFile) {

                var uri = Windows.Foundation.Uri(uriString);
                var downloader = new
                Windows.Networking.BackgroundTransfer.BackgroundDownloader();

                // Create a new download operation.
                download = downloader.createDownload(uri, newFile);

                // Start the download and persist the promise to be able to
                cancel the download.
                promise = download.startAsync().then(complete, error,
                progress);
            }, error);
        } catch (err) {
            displayException(err);
        }
    };

    // On application activation, reassign callbacks for a download
    // operation persisted from previous application state.
    this.load = function (loadedDownload) {
        try {
            download = loadedDownload;
            printLog("Found download: " + download.guid + " from previous
            application run.<br>");
            promise = download.attachAsync().then(complete, error,
            progress);
        } catch (err) {
            displayException(err);
        }
    };
}
```

Note the asynchronous method calls defined using JavaScript promises. Looking at line 17 from the previous code example:

JavaScript

```
promise = download.startAsync().then(complete, error, progress);
```

The async method call is followed by a then statement which indicates methods, defined by the app, that are called when a result from the async method call is returned. For more information on this programming pattern, see [Asynchronous programming in JavaScript using promises](#).

Adding additional operation control methods

The level of control can be increased by implementing additional [DownloadOperation](#) methods. For example, adding the following code to the example above will introduce the ability to cancel the download.

JavaScript

```
// Cancel download.
this.cancel = function () {
    try {
        if (promise) {
            promise.cancel();
            promise = null;
            printLog("Canceling download: " + download.guid + "<br>");
            if (imageStream) {
                imageStream.close();
            }
        }
        else {
            printLog("Download " + download.guid + " already canceled.
<br>");
        }
    } catch (err) {
        displayException(err);
    }
};
```

Enumerating persisted operations at start-up

On completion or cancellation of a [DownloadOperation](#), any associated system resources are released. However, if your app is terminated before either of these events occur, downloads will pause and persist in the background. The following examples demonstrate how to re-introduce persisted downloads into a new app session.

1. Before defining the function that enumerates persisted operations, we need to create an array that will contain the [DownloadOperation](#) objects that it will return:

JavaScript

```
var downloadOps = [];
```

2. Next we define the function that enumerates persisted operations and stores them in our array. Note that the **load** method called to re-assign callbacks for a persisted **DownloadOperation** is in the DownloadOp example we define later in this section.

JavaScript

```
// Enumerate outstanding downloads.
Windows.Networking.BackgroundTransfer.BackgroundDownloader.getCurrentDownloadsAsync().done(function (downloads) {

    for (var i = 0; i < downloads.size; i++) {
        var download = new DownloadOp();
        download.load(downloads[i]);
        downloadOps.push(download);
    }
});
```

3. You can now use the populated list to restart pending operations.

Post-processing

A new feature in Windows 10 is the ability to run application code at the completion of a background transfer even when the app is not running. For example, your app might want to update a list of available movies after a movie has finished downloading, rather than have your app scan for new movies every time it starts. Or your app might want to handle a failed file transfer by trying again using a different server or port. Post-processing is invoked for both successful and failed transfers, so you can use it to implement custom error-handling and retry logic.

Postprocessing uses the existing background task infrastructure. You create a background task and associate it with your transfers before you start the transfers. The transfers are then executed in the background, and when they are complete, your background task is called to perform post-processing.

Post-processing uses a new class, **BackgroundTransferCompletionGroup**. This class is similar to the existing **BackgroundTransferGroup** in that it allows you to group background transfers together, but **BackgroundTransferCompletionGroup** adds the ability to designate a background task to be run when the transfer is complete.

You initiate a background transfer with post-processing as follows.

1. Create a **BackgroundTransferCompletionGroup** object. Then, create a **BackgroundTaskBuilder** object. Set the **Trigger** property of the builder object to the completion group object, and the **TaskEntryPoint** property of the builder to the entry point of the background task that should execute on transfer completion. Finally, call the **BackgroundTaskBuilder.Register** method to register your background task. Note that many completion groups can share one background task entry point, but you can have only one completion group per background task registration.

C#

```
var completionGroup = new BackgroundTransferCompletionGroup();
BackgroundTaskBuilder builder = new BackgroundTaskBuilder();

builder.Name = "MyDownloadProcessingTask";
builder.SetTrigger(completionGroup.Trigger);
builder.TaskEntryPoint = "Tasks.BackgroundDownloadProcessingTask";

BackgroundTaskRegistration downloadProcessingTask = builder.Register();
```

2. Next you associate background transfers with the completion group. Once all transfers are created, enable the completion group.

C#

```
BackgroundDownloader downloader = new BackgroundDownloader(completionGroup);
DownloadOperation download = downloader.CreateDownload(uri, file);
Task<DownloadOperation> startTask = download.StartAsync().AsTask();

// App still sees the normal completion path
startTask.ContinueWith(ForegroundCompletionHandler);

// Do not enable the CompletionGroup until after all downloads are created.
downloader.CompletionGroup.Enable();
```

3. The code in the background task extracts the list of operations from the trigger details, and your code can then inspect the details for each operation and perform appropriate post-processing for each operation.

C#

```
public class BackgroundDownloadProcessingTask : IBackgroundTask
{
    public async void Run(IBackgroundTaskInstance taskInstance)
    {
        var details =
            (BackgroundTransferCompletionGroupTriggerDetails)taskInstance.TriggerDetails
```

```
;
    IReadOnlyList<DownloadOperation> downloads = details.Downloads;

    // Do post-processing on each finished operation in the list of
downloads
    }
}
```

The post-processing task is a regular background task. It is part of the pool of all background tasks, and it is subject to the same resource management policy as all background tasks.

Also, note that post-processing does not replace foreground completion handlers. If your app defines a foreground completion handler, and your app is running when the file transfer completes, then both your foreground completion handler and your background completion handler will be called. The order in which foreground and background tasks are called is not guaranteed. If you define both, you should ensure that the two tasks will work properly and not interfere with each other if they are running concurrently.

Request timeouts

There are two primary connection timeout scenarios to take into consideration:

- When establishing a new connection for a transfer, the connection request is aborted if it is not established within five minutes.
- After a connection has been established, an HTTP request message that has not received a response within two minutes is aborted.

Note In either scenario, assuming there is Internet connectivity, Background Transfer will retry a request up to three times automatically. In the event Internet connectivity is not detected, additional requests will wait until it is.

Debugging guidance

Stopping a debugging session in Microsoft Visual Studio is comparable to closing your app; PUT uploads are paused and POST uploads are terminated. Even while debugging, your app should enumerate and then restart or cancel any persisted uploads. For example, you can have your app cancel enumerated persisted upload operations at app startup if there is no interest in previous operations for that debug session.

While enumerating downloads/uploads on app startup during a debug session, you can have your app cancel them if there is no interest in previous operations for that debug session. Note that if there are Visual Studio project updates, like changes to the app manifest, and the app is uninstalled and re-deployed, [GetCurrentUploadsAsync](#) cannot enumerate operations created using the previous app deployment.

When using Background Transfer during development, you may get into a situation where the internal caches of active and completed transfer operations can get out of sync. This may result in the inability to start new transfer operations or interact with existing operations and [BackgroundTransferGroup](#) objects. In some cases, attempting to interact with existing operations may trigger a crash. This result can occur if the [TransferBehavior](#) property is set to **Parallel**. This issue occurs only in certain scenarios during development and is not applicable to end users of your app.

Four scenarios using Visual Studio can cause this issue.

- You create a new project with the same app name as an existing project, but a different language (from C++ to C#, for example).
- You change the target architecture (from x86 to x64, for example) in an existing project.
- You change the culture (from neutral to en-US, for example) in an existing project.
- You add or remove a capability in the package manifest (adding **Enterprise Authentication**, for example) in an existing project.

Regular app servicing, including manifest updates which add or remove capabilities, do not trigger this issue on end user deployments of your app. To work around this issue, completely uninstall all versions of the app and re-deploy with the new language, architecture, culture, or capability. This can be done via the **Start** screen or using PowerShell and the **Remove-AppxPackage** cmdlet.

Exceptions in Windows.Networking.BackgroundTransfer

An exception is thrown when an invalid string for a Uniform Resource Identifier (URI) is passed to the constructor for the [Windows.Foundation.Uri](#) object.

.NET: The [Windows.Foundation.Uri](#) type appears as [System.Uri](#) in C# and VB.

In C# and Visual Basic, this error can be avoided by using the [System.Uri](#) class in the .NET 4.5 and one of the [System.Uri.TryCreate](#) methods to test the string received from the app user before the URI is constructed.

In C++, there is no method to try and parse a string to a URI. If an app gets input from the user for the [Windows.Foundation.Uri](#), the constructor should be in a try/catch block. If an exception is thrown, the app can notify the user and request a new hostname.

The [Windows.Networking.backgroundTransfer](#) namespace has convenient helper methods and uses enumerations in the [Windows.Networking.Sockets](#) namespace for handling errors. This can be useful for handling specific network exceptions differently in your app.

An error encountered on an asynchronous method in the [Windows.Networking.backgroundTransfer](#) namespace is returned as an **HRESULT** value. The [BackgroundTransferError.GetStatus](#) method is used to convert a network error from a background transfer operation to a [WebErrorStatus](#) enumeration value. Most of the **WebErrorStatus** enumeration values correspond to an error returned by the native HTTP or FTP client operation. An app can filter on specific **WebErrorStatus** enumeration values to modify app behavior depending on the cause of the exception.

For parameter validation errors, an app can also use the **HRESULT** from the exception to learn more detailed information on the error that caused the exception. Possible **HRESULT** values are listed in the *Winerror.h* header file. For most parameter validation errors, the **HRESULT** returned is **E_INVALIDARG**.

Important APIs

- [Windows.Networking.BackgroundTransfer](#)
- [Windows.Foundation.Uri](#)
- [Windows.Networking.Sockets](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Sharing named objects

Article • 11/01/2022

This topic explains how to share named objects between Universal Windows Platform (UWP) applications and Win32 applications.

Named objects in packaged applications

[Named objects](#) provide an easy way for processes to share object handles. After a process has created a named object, other processes can use the name to call the appropriate function to open a handle to the object. Named objects are commonly used for [thread synchronization](#) and [interprocess communication](#).

By default, packaged applications can only access named objects they've created. In order to share named objects with packaged applications, permissions must be set when objects are created, and names must be qualified when objects are opened.

Creating named objects

Named objects are created with a corresponding `Create` API:

- [CreateEvent](#)
- [CreateFileMapping](#)
- [CreateMutex](#)
- [CreateSemaphore](#)
- [CreateWaitableTimer](#)

All of these APIs share an `LPSECURITY_ATTRIBUTES` parameter which enables the caller to specify [access control lists \(ACLs\)](#) to control what processes can access the object. In order to share named objects with packaged applications, permission must be granted within the ACLs when the named objects are created.

Security identifiers (SIDs) represent identities within ACLs. Every packaged application has its own SID based on its package family name. You can generate the SID for a packaged application by passing its package family name to [DeriveAppContainerSidFromAppContainerName](#).

📌 **Note**

The package family name can be found via the package manifest editor in Visual Studio during development time, via **Partner Center** for applications published through the Microsoft Store, or via the **Get-AppxPackage** PowerShell command for applications that are already installed.

This [sample](#) demonstrates the basic pattern needed to ACL a named object. To share named objects with packaged applications, build an **EXPLICIT_ACCESS** structure for each application:

- `grfAccessMode = GRANT_ACCESS`
- `grfAccessPermissions =` appropriate permissions based on the object and intended usage
 - [Generic Access Rights](#)
 - [Synchronization Object Security and Access Rights](#)
 - [File Mapping Security and Access Rights](#)
- `grfInheritance = NO_INHERITANCE`
- `Trustee.TrusteeForm = TRUSTEE_IS_SID`
- `Trustee.TrusteeType = TRUSTEE_IS_USER`
- `Trustee.ptstrName =` the SID acquired from [DeriveAppContainerSidFromAppContainerName](#)

By populating the `LPSECURITY_ATTRIBUTES` parameter in `Create` calls with `EXPLICIT_ACCESS` rules for packaged applications, you can grant access to those applications to open the named object.

ⓘ Note

Win32 applications can access all named objects created by packaged applications as long as they qualify the object names when **opening them**. They do not need to be granted access.

Opening named objects

Named objects are opened by passing a name to a corresponding `Open` API:

- [OpenEvent](#)
- [OpenFileMapping](#)
- [OpenMutex](#)
- [OpenSemaphore](#)
- [OpenWaitableTimer](#)

Named objects created by a packaged application are created within the namespace of the application, otherwise known as the named object path. When opening named objects created by a packaged application, the object names must be prefixed with the creating application's named object path.

[GetAppContainerNamedObjectPath](#) will return the named object path for a packaged application based on its SID. You can generate the SID for a packaged application by passing its package family name to [DeriveAppContainerSidFromAppContainerName](#).

ⓘ Note

The package family name can be found via the package manifest editor in Visual Studio during development time, via **Partner Center** for applications published through the Microsoft Store, or via the **Get-AppxPackage** PowerShell command for applications that are already installed.

When opening named objects created by a packaged application, use the format

`<PATH>\<NAME>`:

- Replace `<PATH>` with the creating application's named object path.
- Replace `<NAME>` with the object name.

ⓘ Note

Prefixing object names with `<PATH>` is only required if a packaged application created the object. Named objects created by Win32 applications do not need to be qualified, though access must still be granted when the objects are **created**.

Remarks

Named objects in packaged applications are isolated by default to preserve security and ensure support for application lifecycle events like suspension and termination. Sharing named objects across applications introduces tight binding and versioning constraints and requires each application to be resilient to the lifecycle of others. For these reasons, it is recommended to only share named objects between applications from the same publisher.

App resources and the Resource Management System

Article • 10/20/2022

This section shows you how to author, package, and consume your app's string, image, and file resources. For example, you might package a file along with your casual game containing a definition of the game's levels, and load the file at run-time. We also show you how maintaining your resources independently of the app's logic makes it easy to localize and customize your app for different locales, device displays, accessibility settings, and other user and machine contexts. Resources such as strings and images typically need to exist in multiple language, scale, and contrast variants. For resources such as that, you have the support of the [Resource Management System](#).

There are two types of app resource.

- A file resource is a resource stored as a file on disk. A file resource can contain a bitmap image, XAML, XML, HTML, or any other kind of data.
- An embedded resource is a resource that is embedded within some containing resource file. The most common example is a string resource embedded within a Resources File (.resw or .resjson).

For more info about the value proposition of localizing your app, see [Globalization and localization](#).

Article	Description
Resource Management System	At build time, the Resource Management System creates an index of all the different variants of the resources that are packaged up with your app. At run-time, the system detects the user and machine settings that are in effect and loads the resources that are the best match for those settings.
How the Resource Management System matches and chooses resources	When a resource is requested, there may be several candidates that match the current resource context to some degree. The Resource Management System will analyze all of the candidates and determine the best candidate to return. This topic describes that process in detail and gives examples.
How the Resource Management System matches language tags	The previous topic (How the Resource Management System matches and chooses resources) looks at qualifier-matching in general. This topic focuses on language-tag-matching in more detail.

Article	Description
Tailor your resources for language, scale, high contrast, and other qualifiers	This topic explains the general concept of resource qualifiers, how to use them, and the purpose of each of the qualifier names.
Localize strings in your UI and app package manifest	If you want your app to support different display languages, and you have string literals in your code or XAML markup or app package manifest, then move those strings into a Resources File (.resw). You can then make a translated copy of that Resources File for each language that your app supports.
Load images and assets tailored for scale, theme, high contrast, and others	Your app can load image resource files containing images tailored for display scale factor, theme, high contrast, and other runtime contexts.
URI schemes	There are several URI (Uniform Resource Identifier) schemes that you can use to refer to files that come from your app's package, your app's data folders, or the cloud. You can also use a URI scheme to refer to strings loaded from your app's Resources Files (.resw).
Specify the default resources that your app uses	If your app doesn't have resources that match the particular settings of a customer device, then the app's default resources are used. This topic explains how to specify what those default resources are.
Build resources into your app package, instead of into a resource pack	Some kinds of apps (multilingual dictionaries, translation tools, etc.) need to override the default behavior of an app bundle, and build resources into the app package instead of having them in separate resource packages. This topic explains how to do that.
Package resource indexing (PRI) APIs and custom build systems	With the package resource indexing (PRI) APIs , you can develop a custom build system for your UWP app's resources. The build system will be able to create, version, and dump (as XML) package resource index (PRI) files to whatever level of complexity your UWP app needs.
Compile resources manually with MakePri.exe	MakePri.exe is a command line tool that you can use to create and dump PRI files. It is integrated as part of MSBuild within Microsoft Visual Studio, but it could be useful to you for creating packages manually or with a custom build system.

Article	Description
Use the Windows 10 Resource Management System in a legacy app or game	By packaging your .NET or Win32 app or game as an .msix or .appx package, you can leverage the Resource Management System to load app resources tailored to the run-time context. This in-depth topic describes the techniques.

Also see [Tile and toast notification support for language, scale, and high contrast](#).

Resource Management System

Article • 10/20/2022

The Resource Management System has both build-time and run-time features. At build time, the system creates an index of all the different variants of the resources that are packaged up with your app. This index is the Package Resource Index, or PRI, and it's also included in your app's package. At run-time, the system detects the user and machine settings that are in effect, consults the information in the PRI, and automatically loads the resources that are the best match for those settings.

Package Resource Index (PRI) file

Every app package should contain a binary index of the resources in the app. This index is created at build time and it is contained in one or more Package Resource Index (PRI) files.

- A PRI file contains actual string resources, and an indexed set of file paths that refer to various files in the package.
- A package typically contains a single PRI file per language, named `resources.pri`.
- The `resources.pri` file at the root of each package is automatically loaded when the [ResourceManager](#) is instantiated.
- PRI files can be created and dumped with the tool [MakePRI.exe](#).
- For typical app development you won't need `MakePRI.exe` because it's already integrated into the Visual Studio compile workflow. And Visual Studio supports editing PRI files in a dedicated UI. However, your localizers and the tools they use might rely upon `MakePRI.exe`.
- Each PRI file contains a named collection of resources, referred to as a resource map. When a PRI file from a package is loaded, the resource map name is verified to match the package identity name.
- PRI files contain only data, so they don't use the portable executable (PE) format. They are specifically designed to be data-only as the resource format for Windows. They replace resources contained within DLLs in the Win32 app model.

UWP API access to app resources

Basic functionality (ResourceLoader)

The simplest way to access your app resources programmatically is by using the [Windows.ApplicationModel.Resources](#) namespace and the [ResourceLoader](#) class.

ResourceLoader provides you basic access to string resources from the set of resource files, referenced libraries, or other packages.

Advanced functionality (ResourceManager)

The **ResourceManager** class (in the [Windows.ApplicationModel.Resources.Core](#) namespace) provides additional info about resources, such as enumeration and inspection. This goes beyond what the **ResourceLoader** class provides.

A **NamedResource** object represents an individual logical resource with multiple language or other qualified variants. It describes the logical view of the asset or resource, with a string resource identifier such as `Header1`, or a resource file name such as `logo.jpg`.

A **ResourceCandidate** object represents a single concrete resource value and its qualifiers, such as the string "Hello World" for English, or "logo.scale-100.jpg" as a qualified image resource that's specific to the **scale-100** resolution.

Resources available to an app are stored in hierarchical collections, which you can access with a **ResourceMap** object. The **ResourceManager** class provides access to the various top-level **ResourceMap** instances used by the app, which correspond to the various packages for the app. The **MainResourceMap** value corresponds to the resource map for the current app package, and it excludes any referenced framework packages. Each **ResourceMap** is named for the package name that is specified in the package's manifest. Within a **ResourceMap** are subtrees (see [ResourceMap.GetSubtree](#)), which further contain **NamedResource** objects. The subtrees typically correspond to the resource files that contains the resource. For more info see [Localize strings in your UI and app package manifest](#) and [Load images and assets tailored for scale, theme, high contrast, and others](#).

Here's an example.

C#

```
// using Windows.ApplicationModel.Resources.Core;
ResourceMap resourceMap =
    ResourceManager.Current.MainResourceMap.GetSubtree("Resources");
ResourceContext resourceContext = ResourceContext.GetForCurrentView()
var str = resourceMap.GetValue("String1", resourceContext).ValueAsString;
```

Note The resource identifier is treated as a Uniform Resource Identifier (URI) fragment, subject to URI semantics. For example, `GetValue("Caption%20")` is treated as `GetValue("Caption ")`. Do not use "?" or "#" in resource identifiers, because they

terminate the resource path evaluation. For example, "MyResource?3" is treated as "MyResource".

The **ResourceManager** not only supports access to an app's string resources, it also maintains the ability to enumerate and inspect the various file resources as well. In order to avoid collisions between files and other resources that originate from within a file, indexed file paths all reside within a reserved "Files" **ResourceMap** subtree. For example, the file `\Images\logo.png` corresponds to the resource name `Files/images/logo.png`.

The **StorageFile** APIs transparently handle references to files as resources, and are appropriate for typical usage scenarios. The **ResourceManager** should only be used for advanced scenarios, such when you want to override the current context.

ResourceContext

Resource candidates are chosen based on a particular **ResourceContext**, which is a collection of resource qualifier values (language, scale, contrast, and so on). A default context uses the app's current configuration for each qualifier value, unless overridden. For example, resources such as images can be qualified for scale, which varies from one monitor to another and hence from one application view to another. For this reason, each application view has a distinct default context. The default context for a given view can be obtained using **ResourceContext.GetForCurrentView**. Whenever you retrieve a resource candidate, you should pass in a **ResourceContext** instance to obtain the most appropriate value for a given view.

Important APIs

- [ResourceLoader](#)
- [ResourceManager](#)
- [ResourceContext](#)

Related topics

- [Localize strings in your UI and app package manifest](#)
- [Load images and assets tailored for scale, theme, high contrast, and others](#)

How the Resource Management System matches and chooses resources

Article • 04/27/2023

When a resource is requested, there may be several candidates that match the current resource context to some degree. The Resource Management System will analyze all of the candidates and determine the best candidate to return. This is done by taking all qualifiers into consideration to rank all of the candidates.

In this ranking process, the different qualifiers are given different priorities: language has the greatest impact on the overall ranking, followed by contrast, then scale, and so on. For each qualifier, candidate qualifiers are compared with the context qualifier value to determine the quality of the match. How the comparison is made depends upon the qualifier.

For specific details on how language tag matching is done, see [How the Resource Management System matches language tags](#).

For some qualifiers, such as scale and contrast, there is always some minimal degree of match. For example, a candidate qualified for "scale-100" matches a context of "scale-400" to some small degree, albeit not as well as a candidate qualified for "scale-200" or (for a perfect match) "scale-400".

For other qualifiers, however, such as language or home region, it is possible to have a non-match comparison (as well as degrees of matching). For example, a candidate qualified for language as "en-US" is a partial match for a context of "en-GB", but a candidate qualified as "fr" is not a match at all. Similarly, a candidate qualified for the home region as "155" (Western Europe) matches a context for a user with a home region setting of "FR" somewhat well, but a candidate qualified as "US" does not match at all.

When a candidate is evaluated, if there is a non-match comparison for any qualifier, then that candidate will get an overall non-match ranking and will not be selected. In this way, the higher-priority qualifiers can have the greatest weight in selecting the best match, but even a low-priority qualifier can eliminate a candidate due to a non-match.

A candidate is neutral in relation to a qualifier if it is not marked for that qualifier at all. For any qualifier, a neutral candidate is always a match for the context qualifier value, but only with a lower quality of match than any candidate that was marked for that qualifier and has some degree of match (exact or partial). For example, if we have candidates qualified for "en-US", "en", "fr", and also a language-neutral candidate, then

for a context with a language qualifier value of "en-GB", the candidates will be ranked in the following order: "en", "en-US", neutral, and "fr". In this case, "fr" does not match at all, while the other candidates match to some degree.

The overall ranking process begins by evaluating candidates in relation to the highest-priority qualifier, which is language. Non-matches are eliminated. The remaining candidates are ranked in relation to their quality of match for language. If there are any ties, then the next-highest-priority qualifier, contrast, is considered, using the quality of match for contrast to differentiate among tied candidates. After contrast, the scale qualifier is used to differentiate remaining ties, and so on, through as many qualifiers as are needed to arrive at a well-ordered ranking.

If all candidates are removed from consideration due to qualifiers that don't match the context, the resource loader goes through a second pass, looking for a default candidate to display. Default candidates are determined during the creation of the PRI file and are required to ensure there is always some candidate to select for any runtime context. If a candidate has any qualifiers that don't match and aren't a default, that resource candidate is thrown permanently out of consideration.

For all the resource candidates still in consideration, the resource loader looks at the highest-priority context qualifier value and chooses the one that has the best match or best default score. Any actual match is considered better than a default score.

If there is a tie, the next-highest priority context qualifier value is inspected, and the process continues until the best match is found.

Example of choosing a resource candidate

Consider these files.

Console

```
en/images/logo.scale-400.jpg
en/images/logo.scale-200.jpg
en/images/logo.scale-100.jpg
fr/images/logo.scale-100.jpg
fr/images/contrast-high/logo.scale-400.jpg
fr/images/contrast-high/logo.scale-100.jpg
de/images/logo.jpg
```

And suppose that these are the settings in the current context.

Console

```
Application language: en-US; fr-FR;  
Scale: 400  
Contrast: Standard
```

The Resource Management System eliminates three of the files because high contrast and the German language do not match the context defined by the settings. That leaves these candidates.

Console

```
en/images/logo.scale-400.jpg  
en/images/logo.scale-200.jpg  
en/images/logo.scale-100.jpg  
fr/images/logo.scale-100.jpg
```

For those remaining candidates, the Resource Management System uses the highest-priority context qualifier, which is language. The English resources are a closer match than the French ones because English is listed before French in the settings.

Console

```
en/images/logo.scale-400.jpg  
en/images/logo.scale-200.jpg  
en/images/logo.scale-100.jpg
```

Next, the Resource Management System uses the next-highest priority context qualifier, scale. So this is the resource returned.

Console

```
en/images/logo.scale-400.jpg
```

You can use the advanced [NamedResource.ResolveAll](#) method to retrieve all of the candidates in the order that they match the context settings. For the example we just walked through, **ResolveAll** returns candidates in this order.

Console

```
en/images/logo.scale-400.jpg  
en/images/logo.scale-200.jpg  
en/images/logo.scale-100.jpg  
fr/images/logo.scale-100.jpg
```

Example of producing a fallback choice

Consider these files.

Console

```
en/images/logo.scale-400.jpg
en/images/logo.scale-200.jpg
en/images/logo.scale-100.jpg
fr/images/contrast-standard/logo.scale-400.jpg
fr/images/contrast-standard/logo.scale-100.jpg
de/images/contrast-standard/logo.jpg
```

And suppose that these are the settings in the current context.

Console

```
User language: de-DE;
Scale: 400
Contrast: High
```

All the files are eliminated because they do not match the context. So we enter a default pass, where the default (see [Compile resources manually with MakePri.exe](#)) during creation of the PRI file was this.

Console

```
Language: fr-FR;
Scale: 400
Contrast: Standard
```

This leaves all the resources that match either the current user or the default.

Console

```
fr/images/contrast-standard/logo.scale-400.jpg
fr/images/contrast-standard/logo.scale-100.jpg
de/images/contrast-standard/logo.jpg
```

The Resource Management System uses the highest-priority context qualifier, language, to return the named resource with the highest score.

Console

```
de/images/contrast-standard/logo.jpg
```

Important APIs

- [NamedResource.ResolveAll](#)

Related topics

- [Compile resources manually with MakePri.exe](#)

How the Resource Management System matches language tags

Article • 10/20/2022

The previous topic ([How the Resource Management System matches and chooses resources](#)) looks at qualifier-matching in general. This topic focuses on language-tag-matching in more detail.

Introduction

Resources with language tag qualifiers are compared and scored based on the app runtime language list. For definitions of the different language lists, see [Understand user profile languages and app manifest languages](#). Matching for the first language in a list occurs before matching of the second language in a list, even for other regional variants. For example, a resource for en-GB is chosen over an fr-CA resource if the app runtime language is en-US. Only if there are no resources for a form of en is a resource for fr-CA chosen (note that the app's default language could not be set to any form of en in that case).

The scoring mechanism uses data that is included in the [BCP-47](#) subtag registry, and other data sources. It allows for a scoring gradient with different qualities of match and, when multiple candidates are available, it selects the candidate with the best-matching score.

So, you can tag language content in generic terms, but you can still specify specific content when needed. For example, your app might have many English strings that are common to both the United States, Britain, and other regions. Tagging these strings as "en" (English) saves space and localization overhead. When distinctions need to be made, such as in a string containing the word "color/colour", the United States and British versions can be tagged separately using both language and region subtags, as "en-US" and "en-GB", respectively.

Language tags

Languages are identified using normalized, well-formed BCP-47 language tags. Subtag components are defined in the BCP-47 subtag registry. The normal structure for a BCP-47 language tag consists of one or more of the following subtag elements.

- Language subtag (required).

- Script subtag (which may be inferred using the default specified in the subtag registry).
- Region subtag (optional).
- Variant subtag (optional).

Additional subtag elements may be present, but they will have a negligible effect on language matching. There are no language ranges defined using the wild card (""), *for example*, "en-".

Matching two languages

Whenever Windows compares two languages it is typically done within the context of a larger process. It may be in the context of assessing multiple languages, such as when Windows generates the application language list (see [Understand user profile languages and app manifest languages](#)). Windows does this by matching multiple languages from the user preferences to the languages specified in the app's manifest. The comparison might also be in the context of assessing language along with other qualifiers for a particular resource. One example is when Windows resolves a particular file resource to a particular resource context; with the user's home location or the device's current scale or dpi as other factors (besides language) that are factored into the resource selection.

When two language tags are compared, the comparison is assigned a score based on the nearness of the match.

Match	Score	Example
Exact match	Highest	en-AU : en-AU
Variant match (language, script, region, variant)		en-AU-variant1 : en-AU-variant1-t-ja
Region match (language, script, region)		en-AU : en-AU-variant1
Partial match (language, script)		
- Macro region match		en-AU : en-053
- Region-neutral match		en-AU : en
- Orthographic affinity match (limited support)		en-AU : en-GB
- Preferred region match		en-AU : en-US
- Any region match		en-AU : en-CA
Undetermined language (any language match)		en-AU : und

Match	Score	Example
No match (script mismatch or primary language tag mismatch)	Lowest	en-AU : fr-FR

Exact match

The tags are exactly equal (all subtag elements match). A comparison may be promoted to this match type from a variant or region match. For example, en-US matches en-US.

Variant match

The tags match on the language, script, region, and variant subtags, but they differ in some other respect.

Region match

The tags match on the language, script, and region subtags, but they differ in some other respect. For example, de-DE-1996 matches de-DE, and en-US-x-Pirate matches en-US.

Partial matches

The tags match on the language and script subtags, but they differ in the region or some other subtag. For example, en-US matches en, or en-US matches en-.*.

Macro region match

The tags match on language and script subtags; both tags have region subtags, one of which denotes a macro region that encompasses the other region. The macro region subtags are always numeric and are derived from the United Nations Statistics Division M.49 country and area codes. For details on encompassing relationships, see [Composition of macro geographic \(continental\) regions, geographical sub-regions, and selected economic and other groupings](#) [↗](#).

Note UN codes for "economic groupings" or "other groupings" are not supported in BCP-47.

Note A tag with the macro-region subtag "001" is considered equivalent to a region-neutral tag. For example, "es-001" and "es" are treated as synonymous.

Region-neutral match

The tags match on language and script subtags, and just one tag has a region tag. A parent match is preferred over other partial matches.

Orthographic affinity match

The tags match on language and script subtags, and the region subtags have orthographic affinity. Affinity relies on data maintained in Windows that defines language-specific affined regions, for example, "en-IE" and "en-GB".

Preferred region match

The tags match on language and script subtags, and one of the region subtags is the default region subtag for the language. For example, "fr-FR" is the default region for the "fr" subtag. So, fr-FR is a better match for fr-BE than is fr-CA. This relies on data maintained in Windows defining a default region for each language in which Windows is localized.

Sibling match

The tags match on language and script subtags, and both have region subtags, but no other relationship is defined between them. In the event of multiple sibling matches, the last enumerated sibling will be the winner, in the absence of a higher match.

Undetermined language

A resource may be tagged as "und" to indicate that it matches any language. This tag may also be used with a script tag to filter matches based on script. For example, "und-Latn" will match any language tag that uses Latin script. See below for more details.

Script mismatch

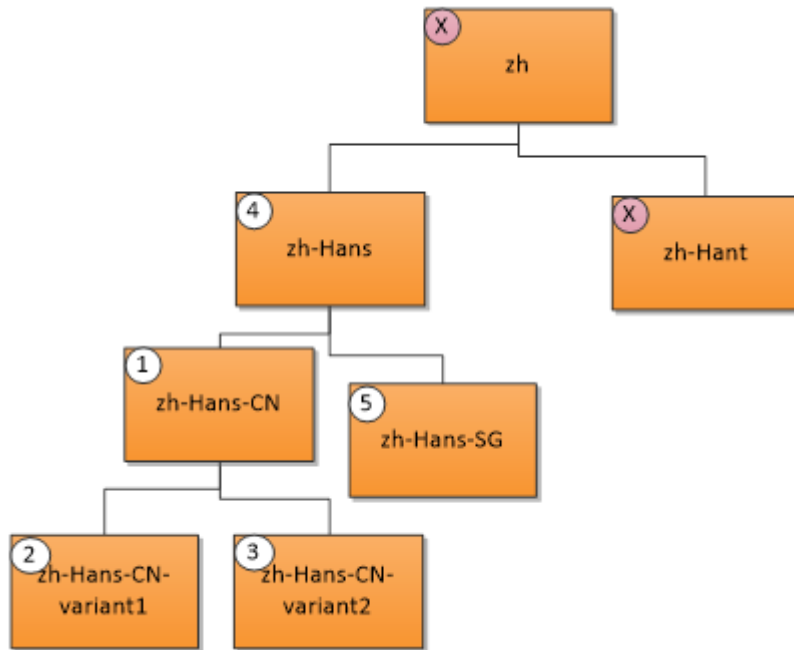
When the tags match only on the primary language tag but not the script, the pair is considered not to match and is scored below the level of a valid match.

No match

Mismatching primary language subtags are scored below the level of a valid match. For example, zh-Hant does not match zh-Hans.

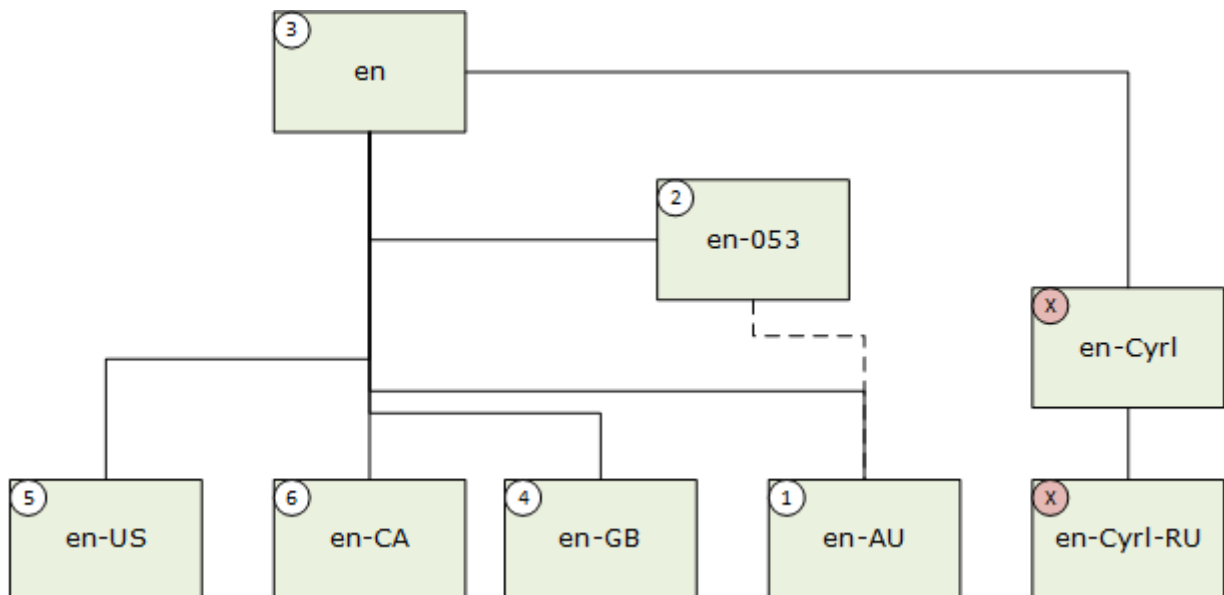
Examples

A user language "zh-Hans-CN" (Chinese Simplified (China)) matches the following resources in the priority order shown. An X indicates no match.



1. Exact match; 2. & 3. Region match; 4. Parent match; 5. Sibling match.

When a language subtag has a Suppress-Script value defined in the BCP-47 subtag registry, corresponding matching occurs, taking on the value of the suppressed script code. For example, en-Latn-US matches en-US. In this next example the user language is "en-AU" (English (Australia)).



1. Exact match; 2. Macro region match; 3. Region-neutral match; 4. Orthographic affinity match; 5. Preferred region match; 6. Sibling match.

Matching a language to a language list

At times, matching occurs as part of a bigger process of matching a single language to a list of languages. For example, there may be a match of a single language-based resource to an app's language list. The score of the match is weighted by the position of the first matching language in the list. The lower the language is in the list, the lower the score will be.

When the language list contains two or more regional variants having the same language and script subtags, comparisons for the first language tag are only scored for exact, variant, and region matches. Scoring partial matches is postponed to the last regional variant. This enables users to finely control the matching behavior for their language list. The matching behavior may include allowing an exact match for a secondary item in the list to be preferred over a partial match for the first item in the list, if there is a third item that matches the language and script of the first. Here's an example.

- Language list (in order): "pt-PT" (Portuguese (Portugal)), "en-US" (English (United States)), "pt-BR" (Portuguese (Brazil)).
- Resources: "en-US", "pt-BR".
- Resource with the higher score: "en-US".
- Description: The comparison starts with "pt-PT" but does not find an exact match. Due to the presence of "pt-BR" in the user's language list, partial matching is postponed to the comparison with "pt-BR". The next language comparison is "en-US", which has an exact match. So, the winning resource is "en-US".

OR

- Language list (in order): "es-MX" (Spanish (Mexico)), "es-HO" (Spanish (Honduras)).
- Resources: "en-ES", "es-HO".
- Resource with the higher score: "es-HO".

Undetermined language ("und")

The language tag "und" may be used to specify a resource that will match any language in the absence of a better match. It can be considered similar to the BCP-47 language range "" or "-<script>". Here's an example.

- Language list: "en-US", "zh-Hans-CN".
- Resources: "zh-Hans-CN", "und".
- Resource with the higher score: "und".

- Description: The comparison starts with "en-US" but does not find a match based on "en" (partial or better). Since there is a resource tagged with "und", the matching algorithm uses that.

The tag "und" enables multiple languages to share a single resource and permits individual languages to be treated as exceptions. For example.

- Language list: "zh-Hans-CN", "en-US".
- Resources: "zh-Hans-CN", "und".
- Resource with the higher score: "zh-Hans-CN".
- Description: The comparison finds an exact match for the first item and so it doesn't check for the resource labeled "und".

You can use "und" with a script tag to filter resources by script. For example.

- Language list: "ru".
- Resources: "und-Latn", "und-Cyrl", "und-Arab".
- Resource with the higher score: "und-Cyrl".
- Description: The comparison doesn't find a match for "ru" (partial or better), and so matches the language tag "und". The suppress-script value "Cyrl" associated with the language tag "ru" matches the resource "und-Cyrl".

Orthographic regional affinity

When two language tags with region subtag differences are matched, particular pairs of regions may have higher affinity to one another than to others. The only supported affined groups are for English ("en"). The region subtags "PH" (Philippines) and "LR" (Liberia) have orthographic affinity with the "US" region subtag. All other region subtags are affined with the "GB" (United Kingdom) region subtag. Therefore, when both "en-US" and "en-GB" resources are available, a language list of "en-HK" (English (Hong Kong SAR)) will get a higher score with "en-GB" resources than with "en-US" resources.

Handling languages with many regional variants

Certain languages have large speaker communities in different regions that use different varieties of that language—languages such as English, French and Spanish, which are among those most often supported in multilingual apps. Regional differences can include differences in orthography (for instance, "color" versus "colour"), or dialect differences such as vocabulary (for instance, "truck" versus "lorry").

These languages with significant regional variants present certain challenges when making a world-ready app: "How many different regional variants should be supported?" "Which ones?" "What's the most cost-effective way to manage these regional variant assets for my app?" It's beyond the scope of this topic to answer all these questions. However, the language matching mechanisms in Windows do provide capabilities that can help you in handling regional variants.

Apps will often support only a single variety of any given language. Suppose an app has resources for just one variety of English that are expected to be used by English speakers regardless of what region they are from. In this case, the tag "en" without any region subtag would reflect that expectation. But apps might have historically used a tag such as "en-US" that includes a region subtag. In this case, that will also work: the app uses only one variety of English, and Windows handles matching a resource tagged for one regional variant with a user language preference for a different regional variant in an appropriate way.

If two or more regional varieties are going to be supported, however, a difference such as "en" versus "en-US" can have a significant impact on the user experience, and it becomes important to consider what region subtags to use.

Suppose you want to provide separate French localizations for French as used in Canada versus European French. For Canadian French, "fr-CA" can be used. For speakers from Europe, the localization will use French (France), and so "fr-FR" can be used for that. But what if a given user is from Belgium, with a language preference of "fr-BE"; which will they get? The region "BE" is different from both "FR" and "CA", suggesting an "any region" match for both. However, France happens to be the preferred region for French, and so the "fr-FR" will be considered the best match in this case.

Suppose you had first localized your app for only one variety of French, using French (France) strings but qualifying them generically as "fr", and then you want to add support for Canadian French. Probably only certain resources need to be re-translated for Canadian French. You can continue to use all the original assets keeping them qualified as "fr", and just add the small set of new assets using "fr-CA". If the user language preference is "fr-CA", then the "fr-CA" asset will have a higher matching score than the "fr" asset. But if the user language preference is for any other variety of French, then the region-neutral asset "fr" will be a better match than the "fr-CA" asset.

As another example, suppose you want to provide separate Spanish localizations for speakers from Spain versus speakers from Latin America. Suppose further that the translations for Latin America were provided from a vendor in Mexico. Should you use "es-ES" (Spain) and "es-MX" (Mexico) for two sets of resources? If you did, that could create problems for speakers from other Latin American regions such as Argentina or

Colombia, since they would get the "es-ES" resources. In this case, there is a better alternative: you can use a macro region subtag, "es-419" to reflect that you intend the assets to be used for speakers from any part of Latin America or the Caribbean.

Region-neutral language tags and macro region subtags can be very effective if you want to support several regional varieties. To minimize the number of separate assets you need, you can qualify a given asset in a way that reflects the broadest coverage for which it is applicable. Then supplement a broadly-applicable asset with a more specific variant as needed. An asset with a region-neutral language qualifier will be used for users of any regional variety unless there is another asset with a more regionally-specific qualifier that applies to that user. For example, an "en" asset will match for an Australian English user, but an asset with "en-053" (English as used in Australia or New Zealand) will be a better match for that user, while an asset with "en-AU" will be the best possible match.

English needs special consideration. If an app adds localization for two English varieties, those will likely be for US English and for UK, or "international", English. As noted above, certain regions outside the US follow United States spelling conventions, and Windows language matching takes that into consideration. In this scenario, it is not recommended to use the region-neutral tag "en" for one of the variants; instead, use "en-GB" and "en-US". (If a given resource doesn't require separate variants, however, "en" can be used.) If either "en-GB" or "en-US" is replaced by "en", then that will interfere with the orthographic regional affinity provided by Windows. If a third English localization is added, then use a specific or macro region subtag for the additional variants as needed (for example, "en-CA", "en-AU" or "en-053"), but continue to use "en-GB" and "en-US".

Related topics

- [How the Resource Management System matches and chooses resources](#)
- [BCP-47](#) ↗
- [Understand user profile languages and app manifest languages](#)
- [Composition of macro geographic \(continental\) regions, geographical sub-regions, and selected economic and other groupings](#) ↗

Tailor your resources for language, scale, high contrast, and other qualifiers

Article • 10/20/2022

This topic explains the general concept of resource qualifiers, how to use them, and the purpose of each of the qualifier names. See [ResourceContext.QualifierValues](#) for a reference table of all the possible qualifier values.

Your app can load assets and resources that are tailored to runtime contexts such as display language, high contrast, [display scale factor](#), and many others. The way you do this is to name your resources' folders or files to match the qualifier names and qualifier values that correspond to those contexts. For example, you may want your app to load a different set of image assets in high contrast mode.

For more info about the value proposition of localizing your app, see [Globalization and localization](#).

Qualifier name, qualifier value, and qualifier

A qualifier name is a key that maps to a set of qualifier values. Here are the qualifier name and qualifier values for contrast.

Context	Qualifier name	Qualifier values
The high contrast setting	contrast	standard, high, black, white

You combine a qualifier name with a qualifier value to form a qualifier. `<qualifier name>-<qualifier value>` is the format of a qualifier. `contrast-standard` is an example of a qualifier.

So, for high contrast, the set of qualifiers is `contrast-standard`, `contrast-high`, `contrast-black`, and `contrast-white`. Qualifier names and qualifier values are not case sensitive. For example, `contrast-standard` and `Contrast-Standard` are the same qualifier.

Use qualifiers in folder names

Here is an example of using qualifiers to name folders that contain asset files. Use qualifiers in folder names if you have several asset files per qualifier. That way, you set the qualifier once at the folder level, and the qualifier applies to everything inside the folder.

Console

```
\Assets\Images\contrast-standard\<logo.png, and other image files>  
\Assets\Images\contrast-high\<logo.png, and other image files>  
\Assets\Images\contrast-black\<logo.png, and other image files>  
\Assets\Images\contrast-white\<logo.png, and other image files>
```

If you name your folders as in the example above, then your app uses the high contrast setting to load resource files from the folder named for the appropriate qualifier. So, if the setting is High Contrast Black, then the resource files in the

`\Assets\Images\contrast-black` folder are loaded. If the setting is None (that is, the computer is not in high contrast mode), then the resource files in the

`\Assets\Images\contrast-standard` folder are loaded.

Use qualifiers in file names

Instead of creating and naming folders, you can use a qualifier to name the resource files themselves. You might prefer to do this if you only have one resource file per qualifier. Here's an example.

Console

```
\Assets\Images\logo.contrast-standard.png  
\Assets\Images\logo.contrast-high.png  
\Assets\Images\logo.contrast-black.png  
\Assets\Images\logo.contrast-white.png
```

The file whose name contains the qualifier most appropriate for the setting is the one that is loaded. This matching logic works the same way for file names as for folder names.

Reference a string or image resource by name

See [Refer to a string resource identifier from XAML markup](#), [Refer to a string resource identifier from code](#), and [Reference an image or other asset from XAML markup and code](#).

Actual and neutral qualifier matches

You don't need to provide a resource file for *every* qualifier value. For example, if you find that you only need one visual asset for high contrast and one for standard contrast,

then you can name those assets like this.

Console

```
\Assets\Images\logo.contrast-high.png  
\Assets\Images\logo.png
```

The first file name contains the `contrast-high` qualifier. That qualifier is an *actual* match for any high contrast setting when high contrast is *on*. In other words, it's a close match so it's preferred. An *actual* match can only occur if the qualifier contains an *actual* value, as this one does. In this case, `high` is an *actual* value for `contrast`.

The file named `logo.png` has no contrast qualifier on it at all. The absence of a qualifier is a *neutral* value. If no preferred match can be found, then the neutral value serves as a fallback match. In this example, if high contrast is *off*, then there is no actual match. The *neutral* match is the best match that can be found, and so the asset `logo.png` is loaded.

If you were to change the name of `logo.png` to `logo.contrast-standard.png`, then the file name would contain an actual qualifier value. With high contrast off, there would be an actual match with `logo.contrast-standard.png`, and that's the asset file that would be loaded. So, the same files would be loaded, under the same conditions, but because of different matches.

If you only need one set of assets for high contrast and one set for standard contrast, then you can use folder names instead of file names. In this case, omitting the folder name entirely gives you the neutral match.

Console

```
\Assets\Images\contrast-high\<logo.png, and other images to load when high  
contrast theme is not None>  
\Assets\Images\<logo.png, and other images to load when high contrast theme  
is None>
```

For more details on how qualifier matching works, see [Resource Management System](#).

Multiple qualifiers

You can combine qualifiers in folder and file names. For example, you may want your app to load image assets when high contrast mode is on *and* the display scale factor is 400. One way to do this is with nested folders.

Console

```
\Assets\Images\contrast-high\scale-400\<logo.png, and other image files>
```

For `logo.png` and the other files to be loaded, the settings must match *both* qualifiers.

Another option is to combine multiple qualifiers in one folder name.

Console

```
\Assets\Images\contrast-high_scale-400\<logo.png, and other image files>
```

In a folder name, you combine multiple qualifiers separated with an underscore.

`<qualifier1>[_<qualifier2>...]` is the format.

You can combine multiple qualifiers in a file name in the same format.

Console

```
\Assets\Images\logo.contrast-high_scale-400.png
```

Depending on the tools and workflow you use for asset-creation, or on what you find easiest to read and/or manage, you can either choose a single naming strategy for all qualifiers, or you can combine them for different qualifiers.

AlternateForm

The `alternateform` qualifier is used to provide an alternate form of a resource for some special purpose. This is typically used only by Japanese app developers to provide a furigana string for which the value `msft-phonetic` is reserved (see the section “Support Furigana for Japanese strings that can be sorted” in [How to prepare for localization](#)).

Either your target system or your app must provide a value against which `alternateform` qualifiers are matched. Do not use the `msft-` prefix for your own custom `alternateform` qualifier values.

Configuration

It’s unlikely that you’ll need the `configuration` qualifier name. It can be used to specify resources that are applicable only to a given authoring-time environment, such as test-only resources.

The `configuration` qualifier is used to load a resource that best matches the value of the `MS_CONFIGURATION_ATTRIBUTE_VALUE` environment variable. So, you can set the variable to the string value that has been assigned to the relevant resources, for example `designer`, or `test`.

Contrast

The `contrast` qualifier is used to provide resources that best match high contrast settings.

Custom

Your app can set a value for the `custom` qualifier, and then resources are loaded that best match that value. For example, you may want to load resources based on your app's license. When your app launches, it checks its license and uses that as the value for the `custom` qualifier by calling [SetGlobalQualifierValue](#), as shown in the code example.

C#

```
public void SetLicenseLevel(BrandID brand)
{
    if (brand == BrandID.Premium)
    {
        ResourceContext.SetGlobalQualifierValue("Custom", "Premium",
        ResourceQualifierPersistence.LocalMachine);
    }
    else if (brand == BrandID.Standard)
    {
        ResourceContext.SetGlobalQualifierValue("Custom", " Standard",
        ResourceQualifierPersistence.LocalMachine);
    }
    else
    {
        ResourceContext.SetGlobalQualifierValue("Custom", "Trial",
        ResourceQualifierPersistence.LocalMachine);
    }
}
```

In this scenario, you would then give your resources names that include the qualifiers `custom-premium`, `custom-standard`, and `custom-trial`.

DeviceFamily

It's unlikely that you'll need the `devicefamily` qualifier name. You can and should avoid using it whenever possible because there are techniques that you can use instead that are much more convenient and robust. Those techniques are described in [Detecting the platform your app is running on](#) and [Version adaptive code](#).

But as a last resort it is possible to use devicefamily qualifiers to name folders that contain your XAML views (a XAML view is a XAML file that contains UI layout and controls).

Console

```
\devicefamily-desktop\<MainPage.xaml, and other markup files to load when
running on a desktop computer>
\devicefamily-mobile\<MainPage.xaml, and other markup files to load when
running on a phone>
```

Or you can name files.

Console

```
\MainPage.devicefamily-desktop.xaml
\MainPage.devicefamily-mobile.xaml
```

In either case each copy of `MainPage.[<qualifier>].xaml` shares a common `MainPage.xaml.cs`, which remains unchanged in your project in terms of name, location, and contents.

You can also use a devicefamily qualifier to name a Resources File (`.resw`), or folder. For example, when your app is running on the mobile device family, the UI element `<TextBlock x:Uid="DeviceFriendlyName"/>` will use the text and foreground resources defined in your `Resources.devicefamily-mobile.resw` file if it contains

XML

```
<data name="DeviceFriendlyName.Foreground">
    <value>Red</value>
</data>
<data name="DeviceFriendlyName.Text">
    <value>Mobile device</value>
</data>
```

For more on using a Resources File, see [Localize your UI strings](#).

DXFeatureLevel

It's unlikely that you'll need the `dxfeaturelevel` qualifier name. It was designed to be used with Direct3D game assets, to cause downlevel resources to be loaded to match a particular downlevel hardware configuration of the time. But the prevalence of that hardware configuration is now so low that we recommend you don't use this qualifier.

HomeRegion

The `homeregion` qualifier corresponds to the user's setting for country or region. It represents the home location of the user. Values include any valid [BCP-47 region tag](#). That is, any **ISO 3166-1 alpha-2** two-letter region code, plus the set of **ISO 3166-1 numeric** three-digit geographic codes for composed regions (see [United Nations Statistic Division M49 composition of region codes](#)). Codes for "Selected economic and other groupings" are not valid.

Language

A `language` qualifier corresponds to the display language setting. Values include any valid [BCP-47 language tag](#). For a list of languages, see the [IANA language subtag registry](#).

If you want your app to support different display languages, and you have string literals in your code or in your XAML markup, then move those strings out of the code/markup and into a Resources File (`.resw`). You can then make a translated copy of that Resources File for each language that your app supports.

You typically use a `language` qualifier to name the folders that contain your Resources Files (`.resw`).

Console

```
\Strings\language-en\Resources.resw
\Strings\language-ja\Resources.resw
```

You can omit the `language-` part of a `language` qualifier (that is, the qualifier name). You can't do this with the other kinds of qualifiers; and you can only do it in a folder name.

Console

```
\Strings\en\Resources.resw
\Strings\ja\Resources.resw
```

Instead of naming folders, you can use `language` qualifiers to name the Resources Files themselves.

Console

```
\Strings\Resources.language-en.resw  
\Strings\Resources.language-ja.resw
```

See [Localize your UI strings](#) for more information on making your app localizable by using string resources, and how to reference a string resource in your app.

LayoutDirection

A `layoutdirection` qualifier corresponds to the layout direction of the display language setting. For example, an image may need to be mirrored for a right-to-left language such as Arabic or Hebrew. Layout panels and images in your UI will respond to layout direction appropriately if you set their `FlowDirection` property (see [Adjust layout and fonts, and support RTL](#)). However, the `layoutdirection` qualifier is for cases where simple flipping isn't adequate, and it allows you to respond to the directionality of specific reading order and text alignment in more general ways.

Scale

Windows automatically selects a scale factor for each display based on its DPI (dots-per-inch) and the viewing distance of the device. See [Effective pixels and scale factor](#). You should create your images at several recommended sizes (at least 100, 200, and 400) so that Windows can either choose the perfect size or can use the nearest size and scale it. So that Windows can identify which physical file contains the correct size of image for the display scale factor, you use a `scale` qualifier. The scale of a resource matches the value of `DisplayInformation.ResolutionScale`, or the next-largest-scaled resource.

Here's an example of setting the qualifier at the folder level.

Console

```
\Assets\Images\scale-100\<logo.png, and other image files>  
\Assets\Images\scale-200\<logo.png, and other image files>  
\Assets\Images\scale-400\<logo.png, and other image files>
```

And this example sets it at the file level.

Console

```
\Assets\Images\logo.scale-100.png  
\Assets\Images\logo.scale-200.png  
\Assets\Images\logo.scale-400.png
```

For info about qualifying a resource for both `scale` and `targetsize`, see [Qualify an image resource for targetsize](#).

TargetSize

The `targetsize` qualifier is primarily used to specify [file type association icons](#) or [protocol icons](#) to be shown in File Explorer. The qualifier value represents the side length of a square image in raw (physical) pixels. The resource whose value matches the View setting in File Explorer is loaded; or the resource with the next-largest value in the absence of an exact match.

You can define assets that represent several sizes of `targetsize` qualifier value for the App Icon (`/Assets/Square44x44Logo.png`) in the Visual Assets tab of the app package manifest designer.

For info about qualifying a resource for both `scale` and `targetsize`, see [Qualify an image resource for targetsize](#).

Theme

The `theme` qualifier is used to provide resources that best match the default app mode setting, or your app's override using [Application.RequestedTheme](#).

Shell light theme and unplated resources

The *Windows 10 May 2019 Update* introduced a new "light" theme for the Windows Shell. As a result, some application assets that were previously shown on a dark background will now be shown on a light background. For apps that provided altform-unplated assets for the taskbar and window switchers (Alt+Tab, Task View, etc), you should verify that they have acceptable contrast on a light background.

Providing light theme specific assets

Apps that want to provide a tailored resource for shell light theme can use a new alternate form resource qualifier: `altform-lightunplated`. This qualifier mirrors the

existing altform-unplated qualifier.

Downlevel considerations

Apps should not use the `theme-light` qualifier with the `altform-unplated` qualifier. This will cause unpredictable behavior on RS5 and earlier versions of Windows due to the way resources are loaded for the Taskbar. On earlier versions of windows, the theme-light version may be used incorrectly. The `altform-lightunplated` qualifier avoids this issue.

Compatibility behavior

For backwards compatibility, Windows includes logic to detect a monochromatic icons and check whether it contrasts with the intended background. If the icon fails to meet contrast requirements, Windows will look for a contrast-white version of the asset. If that's not available, Windows will fall back to using the plated version of the asset.

Important APIs

- [ResourceContext.QualifierValues](#)
- [SetGlobalQualifierValue](#)

Related topics

- [Effective pixels and scale factor](#)
- [Resource Management System](#)
- [How to prepare for localization](#)
- [Detecting the platform your app is running on](#)
- [Programming with extension SDKs](#)
- [Localize your UI strings](#)
- [BCP-47](#) ↗
- [United Nations Statistic Division M49 composition of region codes](#) ↗
- [IANA language subtag registry](#) ↗
- [Adjust layout and fonts, and support RTL](#)

Localize strings in your UI and app package manifest

Article • 06/14/2023

For more info about the value proposition of localizing your app, see [Globalization and localization](#).

If you want your app to support different display languages, and you have string literals in your code or XAML markup or app package manifest, then move those strings into a Resources File (.resw). You can then make a translated copy of that Resources File for each language that your app supports.

Hardcoded string literals can appear in imperative code or in XAML markup, for example as the **Text** property of a **TextBlock**. They can also appear in your app package manifest source file (the `Package.appxmanifest` file), for example as the value for Display name on the Application tab of the Visual Studio Manifest Designer. Move these strings into a Resources File (.resw), and replace the hardcoded string literals in your app and in your manifest with references to resource identifiers.

Unlike image resources, where only one image resource is contained in an image resource file, *multiple* string resources are contained in a string resource file. A string resource file is a Resources File (.resw), and you typically create this kind of resource file in a \Strings folder in your project. For background on how to use qualifiers in the names of your Resources Files (.resw), see [Tailor your resources for language, scale, and other qualifiers](#).

Store strings in a resources file

1. Set your app's default language.
 - a. With your solution open in Visual Studio, open `Package.appxmanifest`.
 - b. On the Application tab, confirm that the Default language is set appropriately (for example, "en" or "en-US"). The remaining steps will assume that you have set the default language to "en-US".

ⓘ Note

At a minimum, you need to provide string resources localized for this default language. Those are the resources that will be loaded if no better match can be found for the user's preferred language or display language settings.

2. Create a Resources File (.resw) for the default language.
 - a. Under your project node, create a new folder and name it `Strings`.
 - b. Under `Strings`, create a new sub-folder and name it `en-US`.
 - c. Under `en-US`, create a new Resources File (.resw) (under the **XAML** file types in the **Add New Item** dialog) and confirm that it is named `Resources.resw`.

ⓘ Note

If you have .NET Resources Files (.resx) that you want to port, see **Porting XAML and UI**.

3. Open `Resources.resw` and add these string resources.

`Strings/en-US/Resources.resw`

Add Resource X Remove Resource			
	Name ▲	Value	Comment
▶	Greeting.Text	Hello	A welcome greeting
	Greeting.Width	20	Width of the welcome greeting
	Farewell	Goodbye	A goodbye
*			

In this example, "Greeting" is a string resource identifier that you can refer to from your markup, as we'll show. For the identifier "Greeting", a string is provided for a Text property, and a string is provided for a Width property. "Greeting.Text" is an example of a property identifier because it corresponds to a property of a UI element. You could also, for example, add "Greeting.Foreground" in the Name column, and set its Value to "Red". The "Farewell" identifier is a simple string resource identifier; it has no sub-properties and it can be loaded from imperative code, as we'll show. The Comment column is a good place to provide any special instructions to translators.

In this example, since we have a simple string resource identifier entry named "Farewell", we cannot *also* have property identifiers based on that same identifier. So, adding "Farewell.Text" would cause a Duplicate Entry error when building `Resources.resw`.

Resource identifiers are case insensitive, and must be unique per resource file. Be sure to use meaningful resource identifiers to provide additional context for translators. And don't change the resource identifiers after the string resources are

sent for translation. Localization teams use the resource identifier to track additions, deletions, and updates in the resources. Changes in resource identifiers—which is also known as "resource identifiers shift"—require strings to be retranslated, because it will appear as though strings were deleted and others added.

Refer to a string resource identifier from XAML

You use an [x:Uid directive](#) to associate a control or other element in your markup with a string resource identifier.

XAML

```
<TextBlock x:Uid="Greeting"/>
```

At run-time, `\Strings\en-US\Resources.resw` is loaded (since right now that's the only Resources File in the project). The `x:Uid` directive on the **TextBlock** causes a lookup to take place, to find property identifiers inside `Resources.resw` that contain the string resource identifier "Greeting". The "Greeting.Text" and "Greeting.Width" property identifiers are found and their values are applied to the **TextBlock**, overriding any values set locally in the markup. The "Greeting.Foreground" value would be applied, too, if you'd added that. But only property identifiers are used to set properties on XAML markup elements, so setting `x:Uid` to "Farewell" on this **TextBlock** would have no effect. `Resources.resw` *does* contain the string resource identifier "Farewell", but it contains no property identifiers for it.

When assigning a string resource identifier to a XAML element, be certain that *all* the property identifiers for that identifier are appropriate for the XAML element. For example, if you set `x:Uid="Greeting"` on a **TextBlock** then "Greeting.Text" will resolve because the **TextBlock** type has a Text property. But if you set `x:Uid="Greeting"` on a **Button** then "Greeting.Text" will cause a run-time error because the **Button** type does not have a Text property. One solution for that case is to author a property identifier named "ButtonGreeting.Content", and set `x:Uid="ButtonGreeting"` on the **Button**.

Instead of setting **Width** from a Resources File, you'll probably want to allow controls to dynamically size to content.

Note For [attached properties](#), you need a special syntax in the Name column of a .resw file. For example, to set a value for the [AutomationProperties.Name](#) attached property for the "Greeting" identifier, this is what you would enter in the Name column.

XML

```
Greeting.[using:Windows.UI.Xaml.Automation]AutomationProperties.Name
```

Refer to a string resource identifier from code

You can explicitly load a string resource based on a simple string resource identifier.

ⓘ Note

If you have a call to any **GetCurrentView** method that *might* be executed on a background/worker thread, then guard that call with an `if`

`(Windows.UI.Core.CoreWindow.GetCurrentThread() != null)` test. Calling **GetCurrentView** from a background/worker thread results in the exception "*<typename> may not be created on threads that do not have a CoreWindow.*"

C#

```
var resourceLoader =  
Windows.ApplicationModel.Resources.ResourceLoader.GetCurrentView();  
this.myXAMLTextBlockElement.Text = resourceLoader.GetString("Farewell");
```

C++/WinRT

```
auto resourceLoader{  
Windows::ApplicationModel::Resources::ResourceLoader::GetCurrentView() };  
myXAMLTextBlockElement().Text(resourceLoader.GetString(L"Farewell"));
```

C++

```
auto resourceLoader =  
Windows::ApplicationModel::Resources::ResourceLoader::GetCurrentView();  
this->myXAMLTextBlockElement->Text = resourceLoader->GetString("Farewell");
```

You can use this same code from within a Class Library (Universal Windows) or a [Windows Runtime Library \(Universal Windows\)](#) project. At runtime, the resources of the app that's hosting the library are loaded. We recommend that a library loads resources from the app that hosts it, since the app is likely to have a greater degree of localization. If a library does need to provide resources then it should give its hosting app the option to replace those resources as an input.

If a resource name is segmented (it contains "." characters), then replace dots with forward slash ("/") characters in the resource name. Property identifiers, for example, contain dots; so you'd need to do this substitution in order to load one of those from code.

C#

```
this.myXAMLTextBlockElement.Text = resourceLoader.GetString("Fare/Well"); //  
<data name="Fare.Well" ...> ...
```

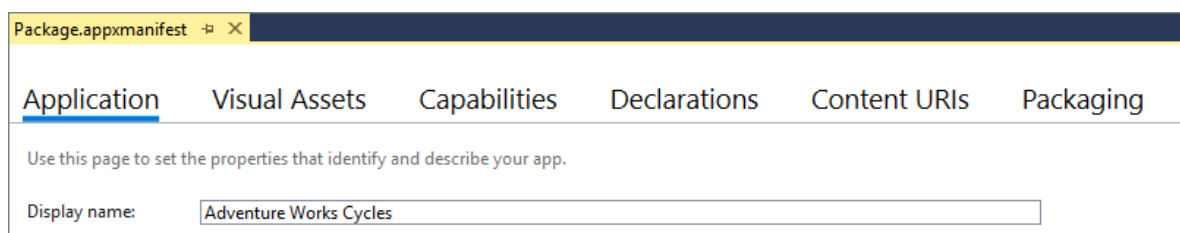
If in doubt, you can use [MakePri.exe](#) to dump your app's PRI file. Each resource's `uri` is shown in the dumped file.

XML

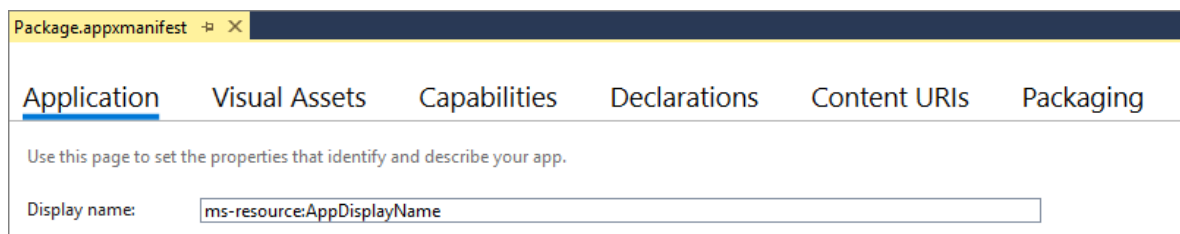
```
<ResourceMapSubtree name="Fare"><NamedResource name="Well" uri="ms-  
resource://<GUID>/Resources/Fare/Well">...
```

Refer to a string resource identifier from your app package manifest

1. Open your app package manifest source file (the `Package.appxmanifest` file), in which by default your app's `Display name` is expressed as a string literal.



2. To make a localizable version of this string, open `Resources.resw` and add a new string resource with the name "AppDisplayName" and the value "Adventure Works Cycles".
3. Replace the Display name string literal with a reference to the string resource identifier that you just created ("AppDisplayName"). You use the `ms-resource` URI (Uniform Resource Identifier) scheme to do this.



4. Repeat this process for each string in your manifest that you want to localize. For example, your app's Short name (which you can configure to appear on your app's tile on Start). For a list of all items in the app package manifest that you can localize, see [Localizable manifest items](#).

Localize the string resources

1. Make a copy of your Resources File (.resw) for another language.
 - a. Under "Strings", create a new sub-folder and name it "de-DE" for Deutsch (Deutschland).

Note For the folder name, you can use any [BCP-47 language tag](#). See [Tailor your resources for language, scale, and other qualifiers](#) for details on the language qualifier and a list of common language tags.
 - b. Make a copy of `Strings/en-US/Resources.resw` in the `Strings/de-DE` folder.
2. Translate the strings.
 - a. Open `Strings/de-DE/Resources.resw` and translate the values in the Value column. You don't need to translate the comments.

`Strings/de-DE/Resources.resw`

Add Resource X Remove Resource			
	Name ▲	Value	Comment
▶	Greeting.Text	Hallo	A welcome greeting
	Greeting.Width	20	Width of the welcome greeting
	Farewell	Auf Wiedersehen	A goodbye
*			

If you like, you can repeat steps 1 and 2 for a further language.

`Strings/fr-FR/Resources.resw`

 Add Resource  Remove Resource			
	Name ▲	Value	Comment
▶	Greeting.Text	Bonjour	A welcome greeting
	Greeting.Width	20	Width of the welcome greeting
	Farewell	Au revoir	A goodbye
*			

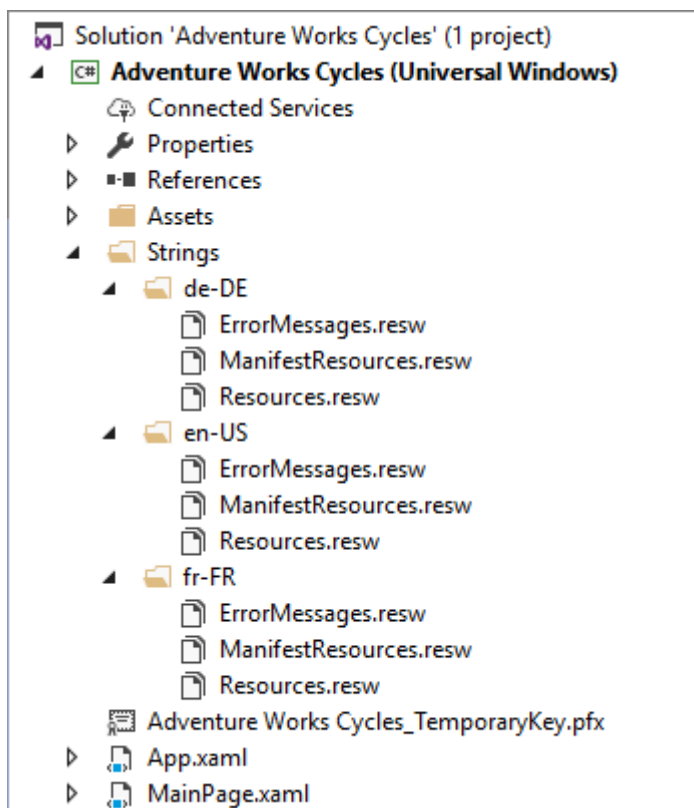
Test your app

Test the app for your default display language. You can then change the display language in **Settings > Time & Language > Region & language > Languages** and re-test your app. Look at strings in your UI and also in the shell (for example, your title bar—which is your Display name—and the Short name on your tiles).

Note If a folder name can be found that matches the display language setting, then the Resources File inside that folder is loaded. Otherwise, fallback takes place, ending with the resources for your app's default language.

Factoring strings into multiple Resources Files

You can keep all of your strings in a single Resources File (resw), or you can factor them across multiple Resources Files. For example, you might want to keep your error messages in one Resources File, your app package manifest strings in another, and your UI strings in a third. This is what your folder structure would look like in that case.



To scope a string resource identifier reference to a particular file, you just add `/<resources-file-name>/` before the identifier. The markup example below assumes that `ErrorMessages.resw` contains a resource whose name is "PasswordTooWeak.Text" and whose value describes the error.

XAML

```
<TextBlock x:Uid="/ErrorMessages/PasswordTooWeak"/>
```

You only need to add `/<resources-file-name>/` before the string resource identifier for Resources Files *other than* `Resources.resw`. That's because "Resources.resw" is the default file name, so that's what's assumed if you omit a file name (as we did in the earlier examples in this topic).

The code example below assumes that `ErrorMessages.resw` contains a resource whose name is "MismatchedPasswords" and whose value describes the error.

ⓘ Note

If you have a call to any **GetCurrentView** method that *might* be executed on a background/worker thread, then guard that call with an `if`

`(Windows.UI.Core.CoreWindow.GetCurrentThread() != null)` test. Calling

GetCurrentView from a background/worker thread results in the exception "`<typename> may not be created on threads that do not have a CoreWindow.`"

C#

```
var resourceLoader =  
Windows.ApplicationModel.Resources.ResourceLoader.GetForCurrentView("ErrorMe  
ssages");  
this.myXMLTextBlockElement.Text =  
resourceLoader.GetString("MismatchedPasswords");
```

C++/WinRT

```
auto resourceLoader{  
Windows::ApplicationModel::Resources::ResourceLoader::GetForCurrentView(L"Er  
rorMessages") };  
myXMLTextBlockElement().Text(resourceLoader.GetString(L"MismatchedPasswords  
"));
```

C++

```
auto resourceLoader =  
Windows::ApplicationModel::Resources::ResourceLoader::GetForCurrentView("Err  
orMessages");  
this->myXMLTextBlockElement->Text = resourceLoader->  
GetString("MismatchedPasswords");
```

If you were to move your "AppDisplayName" resource out of `Resources.resw` and into `ManifestResources.resw`, then in your app package manifest you would change `ms-resource:AppDisplayName` to `ms-resource:/ManifestResources/AppDisplayName`.

If a resource file name is segmented (it contains "." characters), then leave the dots in the name when you reference it. **Don't** replace dots with forward slash ("/") characters, like you would for a resource name.

C#

```
var resourceLoader =  
Windows.ApplicationModel.Resources.ResourceLoader.GetForCurrentView("Err.Msg  
s");
```

If in doubt, you can use [MakePri.exe](#) to dump your app's PRI file. Each resource's `uri` is shown in the dumped file.

XML

```
<ResourceMapSubtree name="Err.Msgs"><NamedResource  
name="MismatchedPasswords" uri="ms-  
resource://<GUID>/Err.Msgs/MismatchedPasswords">...
```